

T09 : Develop a Generative Model for Creating Personalized Marketing Content

1. Introduction

This project focuses on building a Generative Model for creating personalized marketing emails using the Enron Email Dataset. The model is built using Long Short-Term Memory (LSTM) networks with Bidirectional layers to enhance the model's ability to generate contextually relevant emails.

Objectives:

- Train a generative model on the Enron Email Dataset.
 - Use LSTM architecture for text generation.
 - Generate realistic and coherent marketing email text.
 - Evaluate the model's performance and fine-tune its generation capabilities.
-

2. Dataset Description & Preprocessing

2.1. Dataset Overview

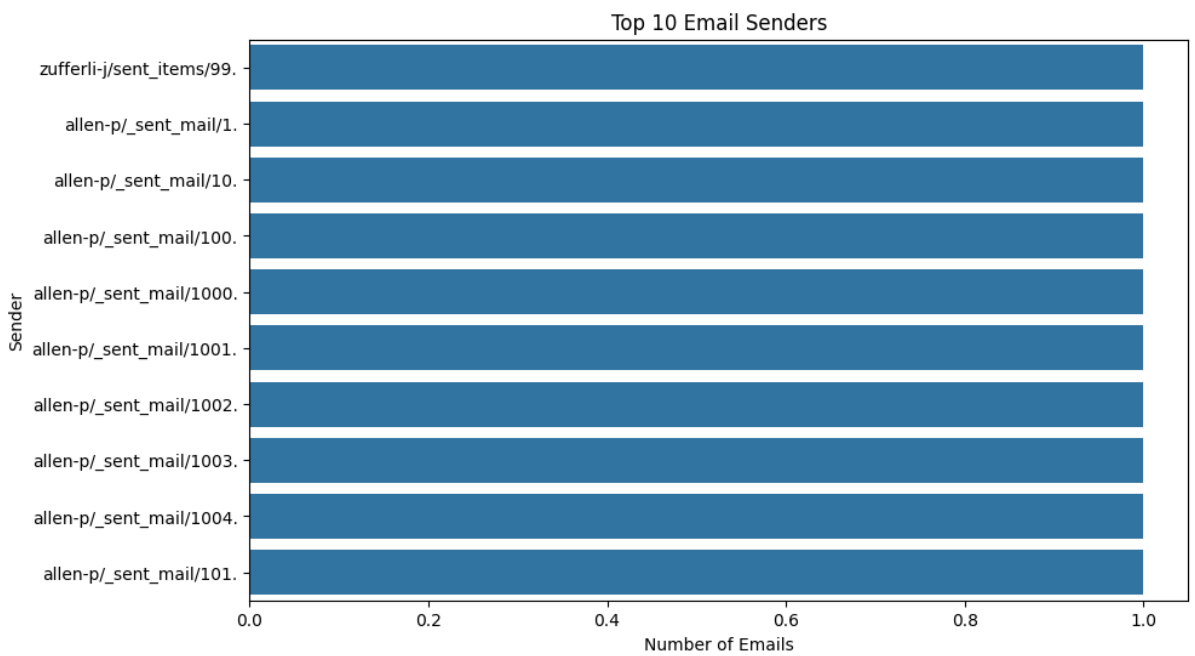
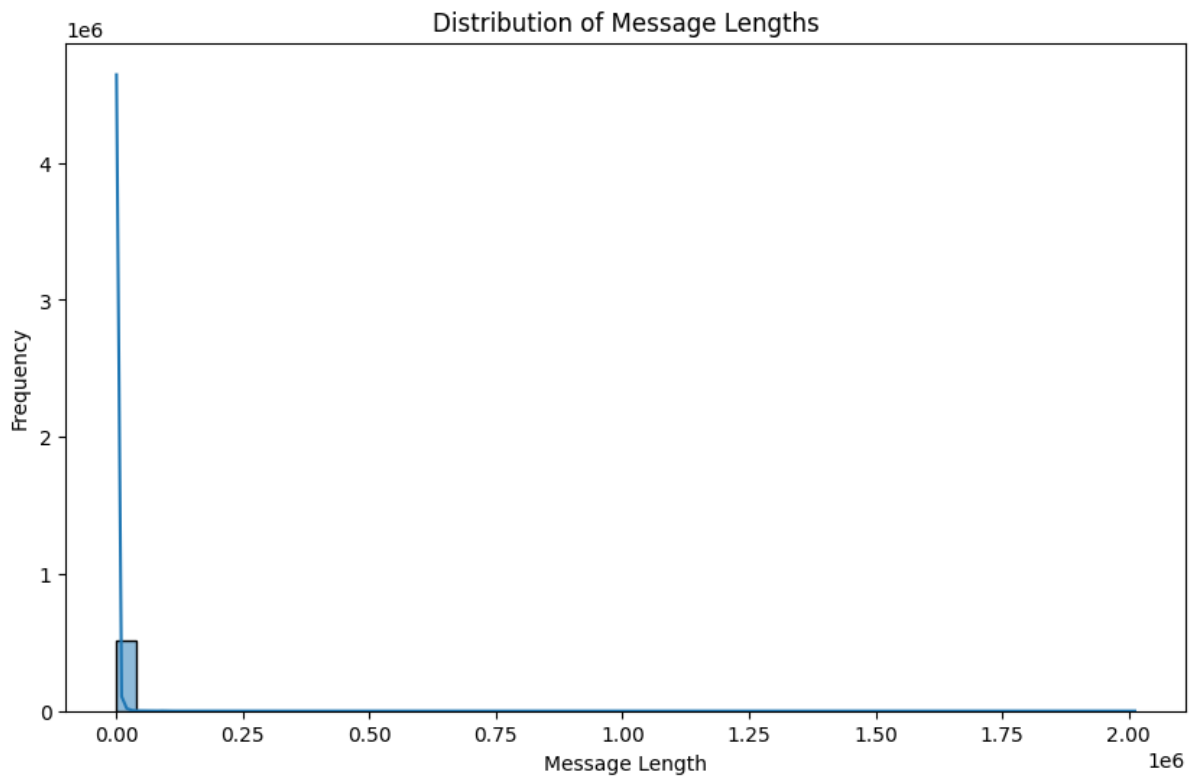
The Enron Email Dataset is a collection of real emails sent by Enron Corporation employees. For this project:

- **Corpus Size:** 399,950 email sequences.
 - **Dataset Shape:**
 - Shape of X: (399950, 50, 1)
 - Shape of y: (399950, 29)
-

2.2. Data Preprocessing

To prepare the data:

- **Text Cleaning:** Removed unnecessary characters, and punctuation, and standardized the case.
 - **Data Transformation:** Each email is split into sequences of 50 characters, and the target is the next character in the sequence.
 - **Normalization & Encoding:**
 - Character to integer mapping for input sequences.
 - One-hot encoding for target sequences (next character prediction).
-



3. Model Selection & Implementation

The model uses an LSTM-based architecture with the following layers:

- Bidirectional LSTM Layer: For capturing both past and future context.
- Dropout: For regularization.
- Dense Layer: For predicting the next character using softmax activation.

Model Summary:

- Total Parameters: 5,267,997
- Bidirectional LSTM Layer: 2,105,344 parameters
- LSTM Layer: 3,147,776 parameters
- Dense Layer: 14,877 parameters

Model architecture visualization:

Model: "sequential"

Layer (type)	Output Shape	Param #
bidirectional (Bidirectional)	(None, 50, 1024)	2,105,344
dropout (Dropout)	(None, 50, 1024)	0
lstm_1 (LSTM)	(None, 512)	3,147,776
dropout_1 (Dropout)	(None, 512)	0
dense (Dense)	(None, 29)	14,877

Total params: 5,267,997 (20.10 MB)

Trainable params: 5,267,997 (20.10 MB)

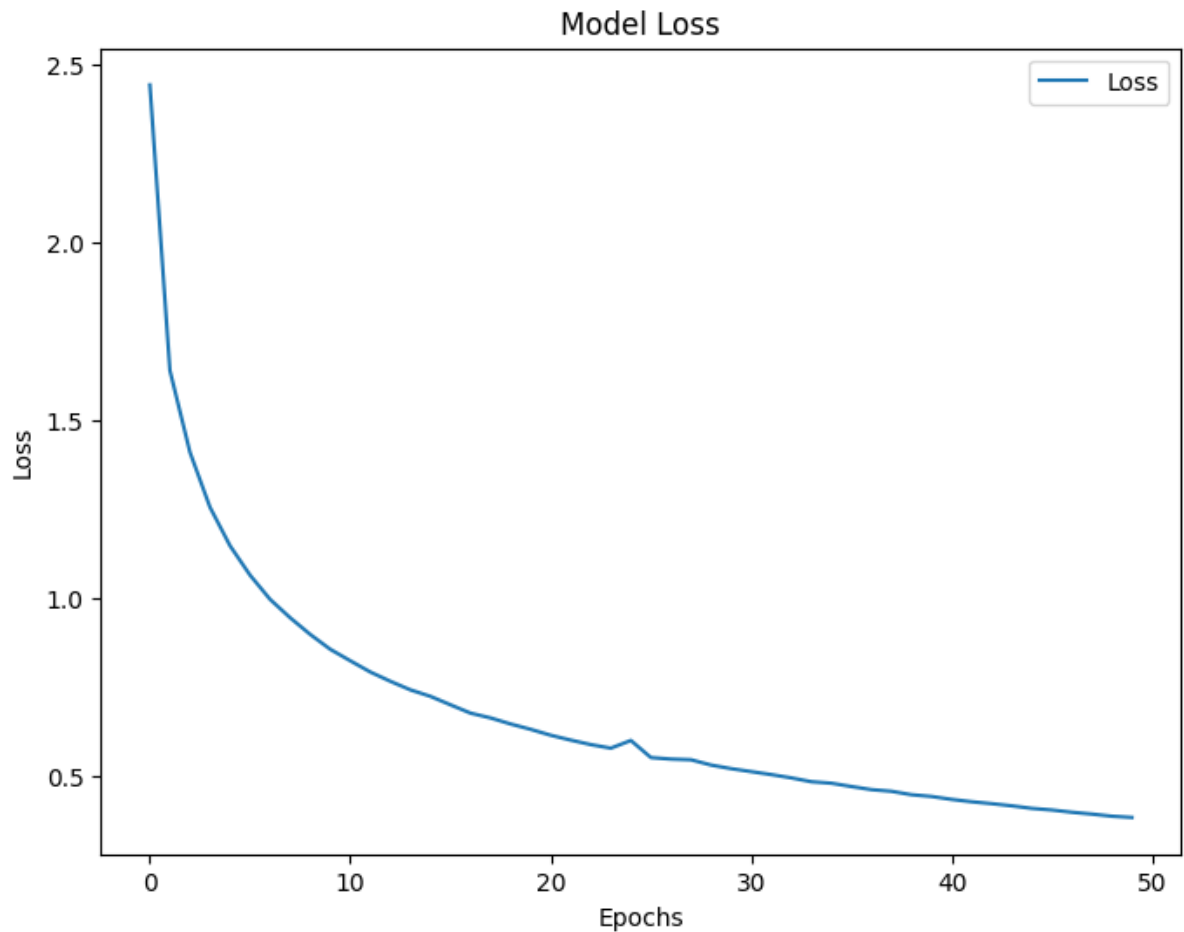
Non-trainable params: 0 (0.00 B)

4. Model Training & Evaluation

4.1 Training Overview

- The model was trained for 50 epochs using a batch size of 128. The training loss gradually decreased, and the model achieved a final training loss of 0.45.

Training Loss Graph:



4.2 Performance Metrics

After training, the model was evaluated on a test dataset. The test loss and perplexity were calculated as:

- Test Loss: 0.17
- Perplexity: 1.19

```
Test Loss: 0.17439435422420502
Perplexity: 1.1905249618078446
```

5. Generated Text Samples

5.1. Post Processing Generated Text

After generating text, the postprocessing function was applied to correct incomplete phrases and sentence fragments. Here's an example of the postprocessed output:

Generated Text (before postprocessing):

```
# Test the model with a seed text
seed_text = "hello, i hope"
generated_text = generate_text(model, seed_text)
print("Generated Text:\n", generated_text)
```

Generated Text:

```
hello, i hope mentia messageid javamailevansthyme
date tue jul pdt
from phillipallenenroncom
to jsmithaustintxcom
subject re the stage
mimeversion
contenttype textplain charsetusascii
contenttransferencoding bit
xfrom phillip k allen
xto mary richards maryrichardshotmailcom enron
xcc
xbcc
xfolder phillipallendecnotes folders sent mail
xorigin allenp
xfilename pallennsf

forwarded by phillip k allenhouect on
pm

enron north america corp
```

Postprocessed Text:

```
# Test postprocessing on the generated text
generated_text = "hello, i hope your of the loan remains in the builderdevelopers name"
processed_text = postprocess_text(generated_text)
print(processed_text)
```

hello, i hope your of the loan remains in the builderdevelopers name.

5.2 Temperature-based Sampling

The model's output was generated with different temperature values, affecting the randomness of the text generation.

Temperature = 0.2: Generates very predictable text.

```
Generated text with temperature 0.2:
hello, i hope mentia messageid javamailevansthyme
date tue apr pdt
from phillipallenroncom
to jsmithaustintxcom
subject re resource
mimeversion
contenttype textplain charsetusascii
contenttransferencoding b
```

Temperature = 0.5: A balance between randomness and predictability.

```
Generated text with temperature 0.5:
hello, i hope ment m presenteoroenm

for to thould you please expect that the republicans are not
sure it wery select that was prepared in the west desk
mimeversion
contenttype textplain charsetusascii
contenttr
```

Temperature = 1.0: Generates more creative and diverse text.

```
Generated text with temperature 1.0:
hello, i hope y
can you help
please advise of somiing with them more gas
cetailed
phillip messageid javamailevansthyme
date tue jul pdt
from phillipallenroncom
to pallenhotmailcom
subject re strprtruig
```

Generated Text with Different Temperatures:

```
Generated text with seed 'Subject: Meeting':
Subject: Meeting so be pay have in the safe side the engineer in schulatory risk on the
load side habasis market this is a xould be better for a rurchaladds with the but we are internet explorer and in the extr
```

Sample Output:

```
# Test the model with a seed text and temperature
generated_text = generate_text_with_temperature(model, seed_text="hello, i hope", num_chars=500, temperature=0.8)
print(generated_text)

hello, i hope ment be billion day gavidir in grnwe be in linked to aeoron gor a than is notiin might be the addition of a configured by the buciagst o
wear ken wants to have this updated chart in his briefing book for nessageid javamailevansthyme
date fri jun pdt
from phillipallenroncom
to stouchstonenatsourcecom
subject re unom ass nearian telleniogs
mimeversion
contenttype textplain charsetusascii
contenttransferencoding
```

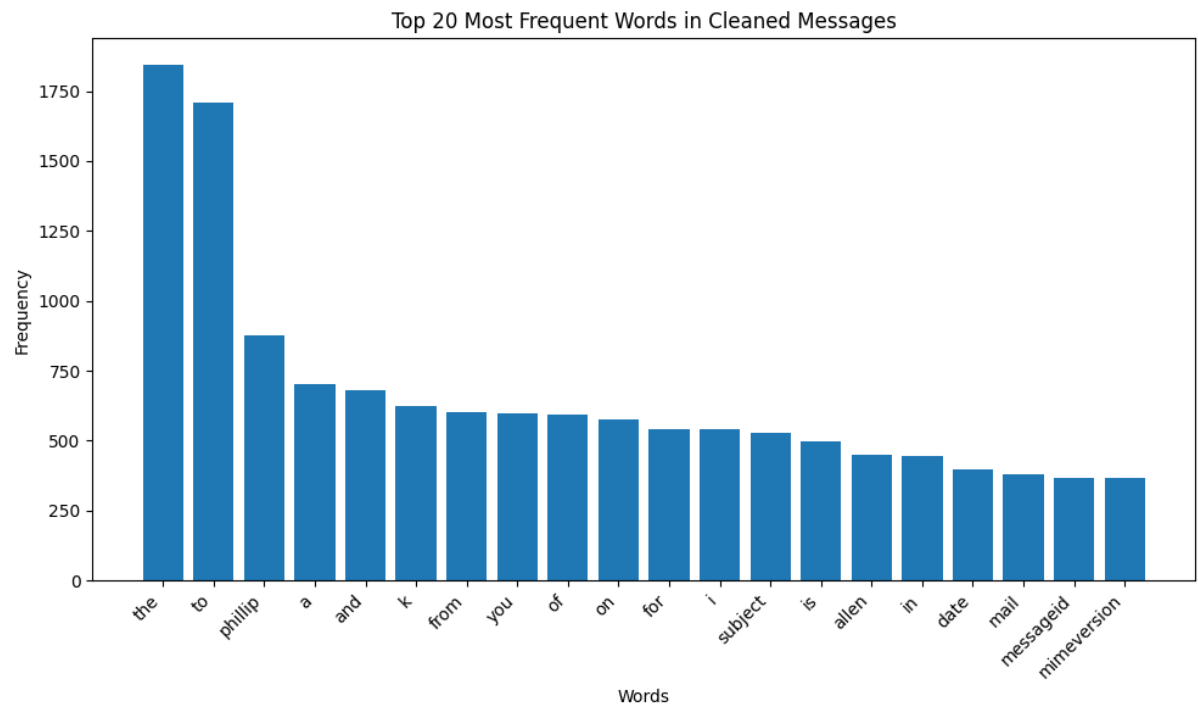
6. Results & Analysis

6.1 Evaluation of Generated Text

The generated emails mostly retained their structure but had incoherent fragments or missing information. The postprocessing helped improve these issues, although some text was still nonsensical.

```
Generated text with seed 'Subject: Meeting':
Subject: Meeting so be pay have in the safe side the engineer in schulatory risk on the
load side habasis market this is a xould be better for a rurchaladds with the but we are internet explorer and in the extr
```

6.2 Top 20 Most Frequent Words in Cleaned Messages



The Generative AI model for marketing emails shows promising results. While the text generated is largely coherent, improvements in postprocessing and fine-tuning are needed for more realistic results. Future work will focus on refining the model for better performance and real-world deployment.